RENT Right

استأجر صح

Supervised by:

Prof. Ahmed Alkhodre

Mahdi Abulkhair [redacted]
Mohammed Abdulsamed [redacted]

Table Of Contents

1. Problem Statement

Today, people often spend time with friends and family by going on picnics, camping trips, or other outdoor activities. These outings usually need certain items like lamps, grills, or small electrical devices. Most of the time, these things are only needed briefly.

Because of this, people often end up buying items they may use only once or a few times. This leads to unnecessary spending and wasted money, especially for users who do not need these items regularly. It can also be inconvenient to store items that are rarely used.

At the same time, many people already own such items but do not use them frequently. These items often remain unused at home, taking up space without providing any real value. This creates a gap where some people need items, while others have them but are not using them.

Currently, there is no simple and effective way for people to connect and share these resources. As a result, both sides miss out on potential benefits–users spend more money than necessary, while item owners do not take advantage of their unused belongings.

Therefore, there is a need for a system that allows people to easily rent and share items. This system would help users access what they need at a lower cost, while also allowing others to benefit from items they already own. In this way, resources can be used more efficiently, and unnecessary spending can be reduced.

2. Requirement Specification

2.1 The functional requirements .....

The system shall allow the users to ***add listing***

The system shall allow users to ***browse available listings.***

The system shall allow users to ***search for a listing.***

The system shall allow users to ***view nearby listings***.

The system shall allow users to lease or ***book a listed item.***

The system shall allow users to ***send messages to the lessor***.

The system shall allow users to ***view their profile.***

The system shall allow users to ***rate the lessor after leasing***.

The system shall allow the Renter to ***pay insurance***

The system shall ***transfer the insurance fee*** to the lessor if the listing is damaged.

2.2 The Non functional requirements .....

- **Performance**: For Browse Listings and Search for a Listing, the system **shall load results within less than 2 seconds.** For Book Item and Register/Login, operations shall be completed within 2 seconds to ensure a smooth user experience.
- **Usability**: For Add Listing, Book Item, and Send Messages, the system shall provide a **simple and user-friendly interface.** Users shall be able to complete tasks through clear and minimal steps.
- **Security**: For Register/Login, the system **shall implement secure authentication mechanisms.** For View Profile, user data must be protected from unauthorized access. For Pay Insurance, all payment transactions shall be encrypted.
- **Reliability**: For Book Item and Pay Insurance, **the system shall ensure operations are completed accurately without errors or duplication.** Data loss must be prevented during transactions.
- **Availability**: For Send Messages, Browse Listings, and Book Item, **the system shall be available 24/7** without interruption.
- **Scalability**: For Browse Listings and Search for a Listing, the system **shall handle a large number of users** and listings without performance degradation.

Glossary, and Assumptions

2.3 Glossary of Terms

The following terms are used in this document:

System: The Rent Right platform including the application, server, and database

User: Any person using the system (Renter, Lessor, or Admin).

Renter: A user who searches and books items.

Lessor: A user who adds items for rent.

Admin: A user who manages the system and resolves issues.

Listing: An item available for rent.

Booking: Reserving an item for a specific period.

Payment: The process of paying rental and insurance fees.

Insurance: A fee paid to cover possible damage.

Report: A complaint or damage report submitted by users.

Review: Feedback given after completing a booking.

SRD: Software Requirements Document.

RTM: Requirements Traceability Matrix.

2.4 Assumptions and Dependencies

The system is designed based on the following assumptions:

Users have access to the internet while using the system.

Each user has a valid account and correct information.

The system uses an external service for payment processing.

Each item belongs to one lessor only.

A booking cannot be made if the item is already reserved.

Users provide accurate item details and images.

Payments are processed successfully unless an error occurs.

Reviews are added only after completing a booking.

The admin is responsible for managing users and resolving issues.

Location features depend on user location availability.

2.5 Requirements Traceability Matrix (RTM)

Req ID	Requirement	Type	Use Case	Class	Methods
FR-01	Add listing	Functional	Add Listing	Item, User	***addItem()***
FR-02	Browse listings	Functional	Browse Listings	Item	***viewDetails()***
FR-03	Search for a listing	Functional	Search Item	Item	***viewDetails()***
FR-04	View nearby listings	Functional	View Nearby Listings	Item	***viewDetails()***
FR-05	Book item	Functional	Book Item	Booking, User	***createBooking(), bookItem()***
FR-06	Send messages	Functional	Send Messages	Message, User	***sendMessage()***
FR-07	View profile	Functional	View Profile	User	***viewProfile()***
FR-08	Rate lessor	Functional	Add Review	Review, User	***addReview()***
FR-09	Pay insurance	Functional	Pay Insurance	Payment, User	***makePayment(), processPayment()***
FR-10	Transfer insurance fee	Functional	Transfer Insurance	Payment, Admin	***transferInsurance()***
NFR-01	Load results < 2 sec	Performance	Search / Browse	Item	***viewDetails()***
NFR-02	Fast booking/login	Performance	Book Item / Login	Booking, User	***createBooking(), login()***
NFR-03	User-friendly UI	Usability	All	UI Layer	—
NFR-04	Secure authentication	Security	Register/Login	User	***register(), login()***
NFR-06	Accurate booking	Reliability	Book Item	Booking	***confirmBooking(), rejectBooking()***
NFR-07	24/7 availability	Availability	All	Server	—
NFR-08	Handle many users	Scalability	Search/Browse	System	—

3. System Architecture Specification

Client-Server Architecture is the Architecture that we used in Rent Right Project

Client-Server is a distributed architectural pattern where the system is divided into **two main roles**: a Client that requests services, and a Server that provides them. The client and server communicate over a network through a defined protocol

According to Chapter 4 (Dr. ALKHODRE), this pattern is used when data in a shared database has to be accessed from a range of locations, and when the load on a system is variable – both of which apply directly to **Rent Right** .

3.1 How It Applies to Rent Right ?

Who is the Client? the application that on the user's device. This includes three types of users:

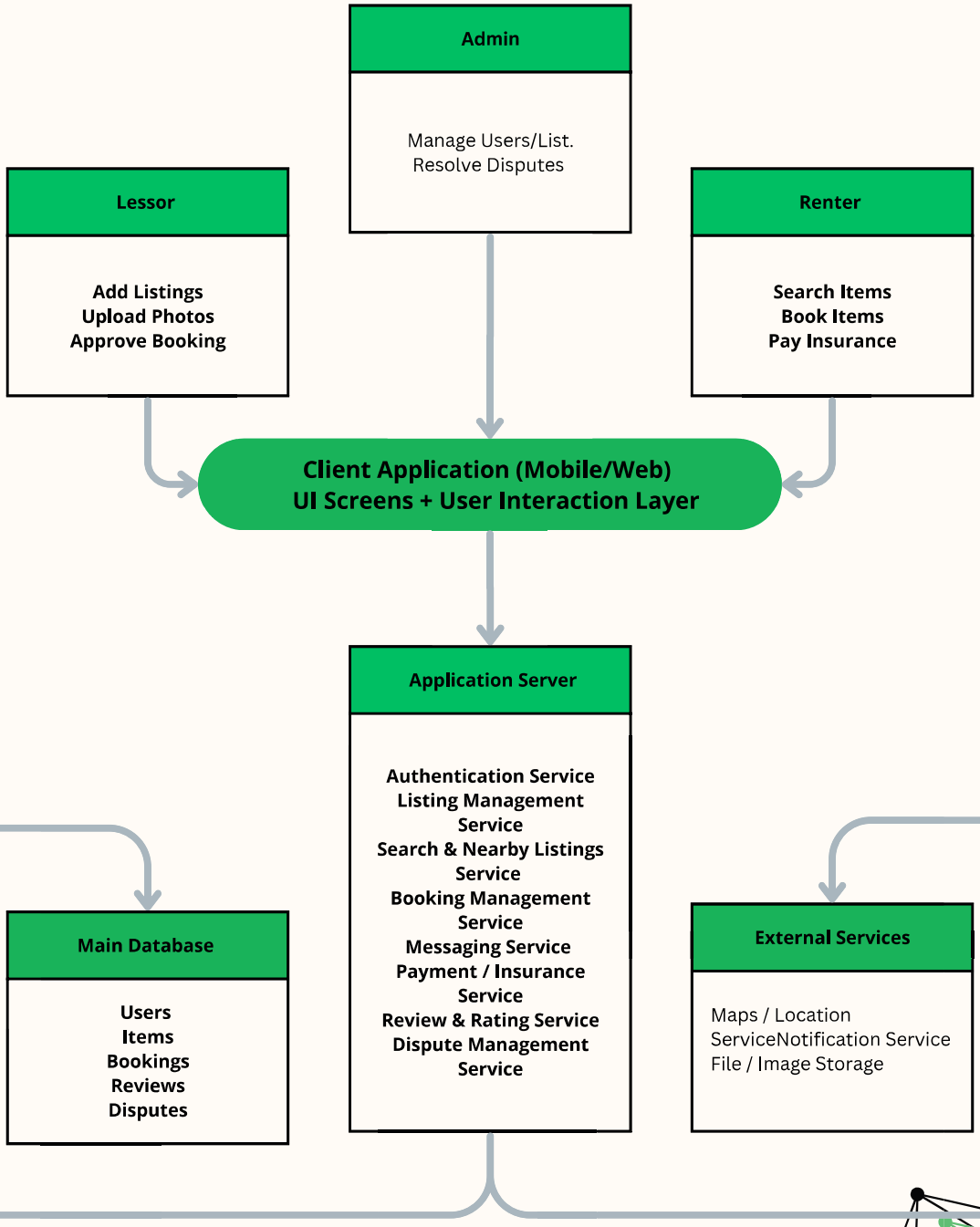
- Lessor – lists items, uploads photos, approves booking requests.
- Renter – searches for items, books them, pays rental fee and deposit.
- Admin – monitors the system and resolves disputes

Who is the Server? The system in the cloud. It receives requests from the application and handles all business logic like :

- Authentication
- Booking – checks availability and schedules rentals.
- Guarantee calculator – computes deposit based on item value and condition.
- automatic contract generation for each rental transaction
- Dispute management – receives evidence and supports admin decisions.
- Review service – stores and computes user trust ratings..

The Database The central database contains six tables: Users, Items, Bookings, Payments, Reviews, and Disputes – all accessed only through the server.

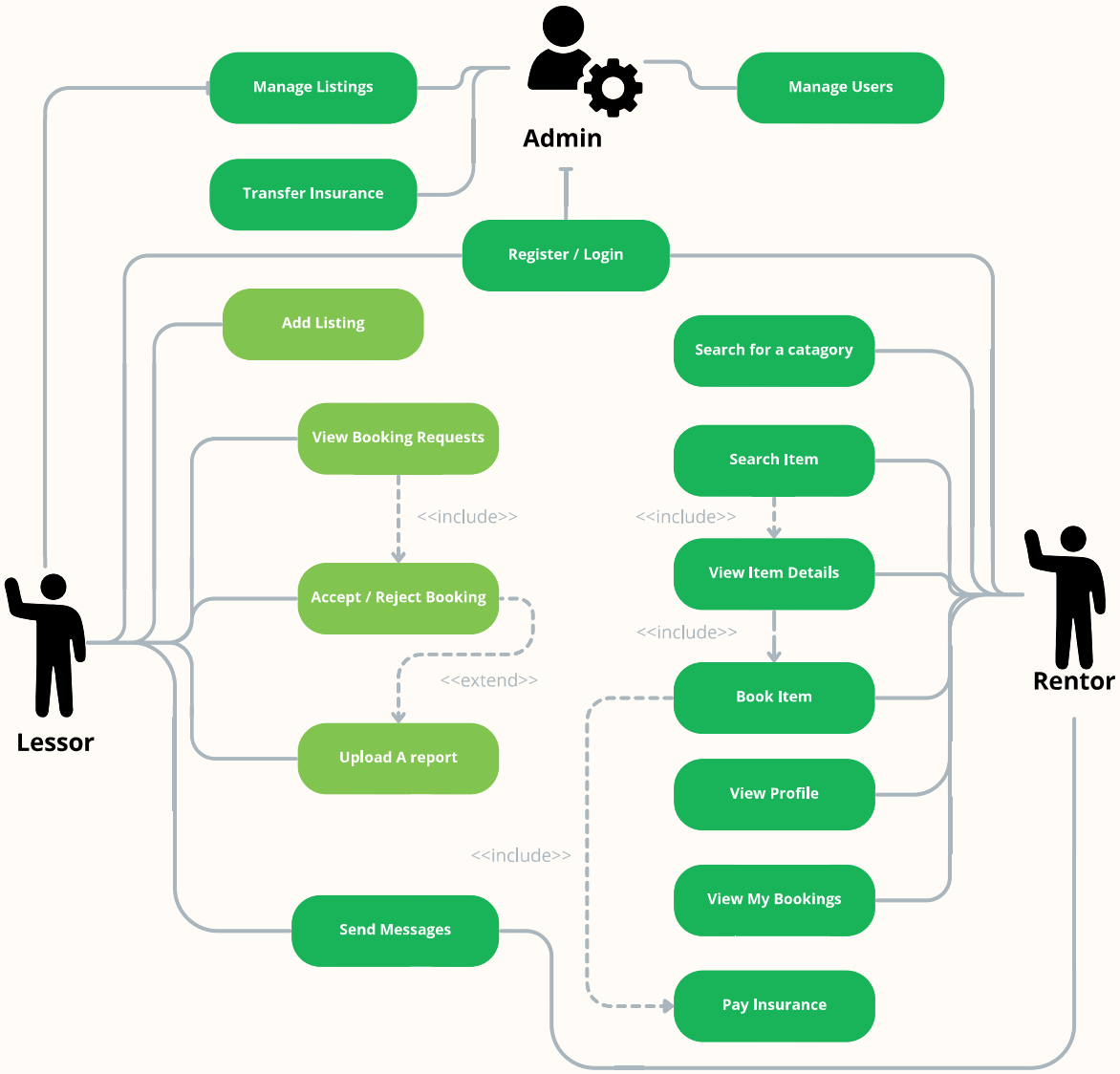
3.2 The Architecture Design



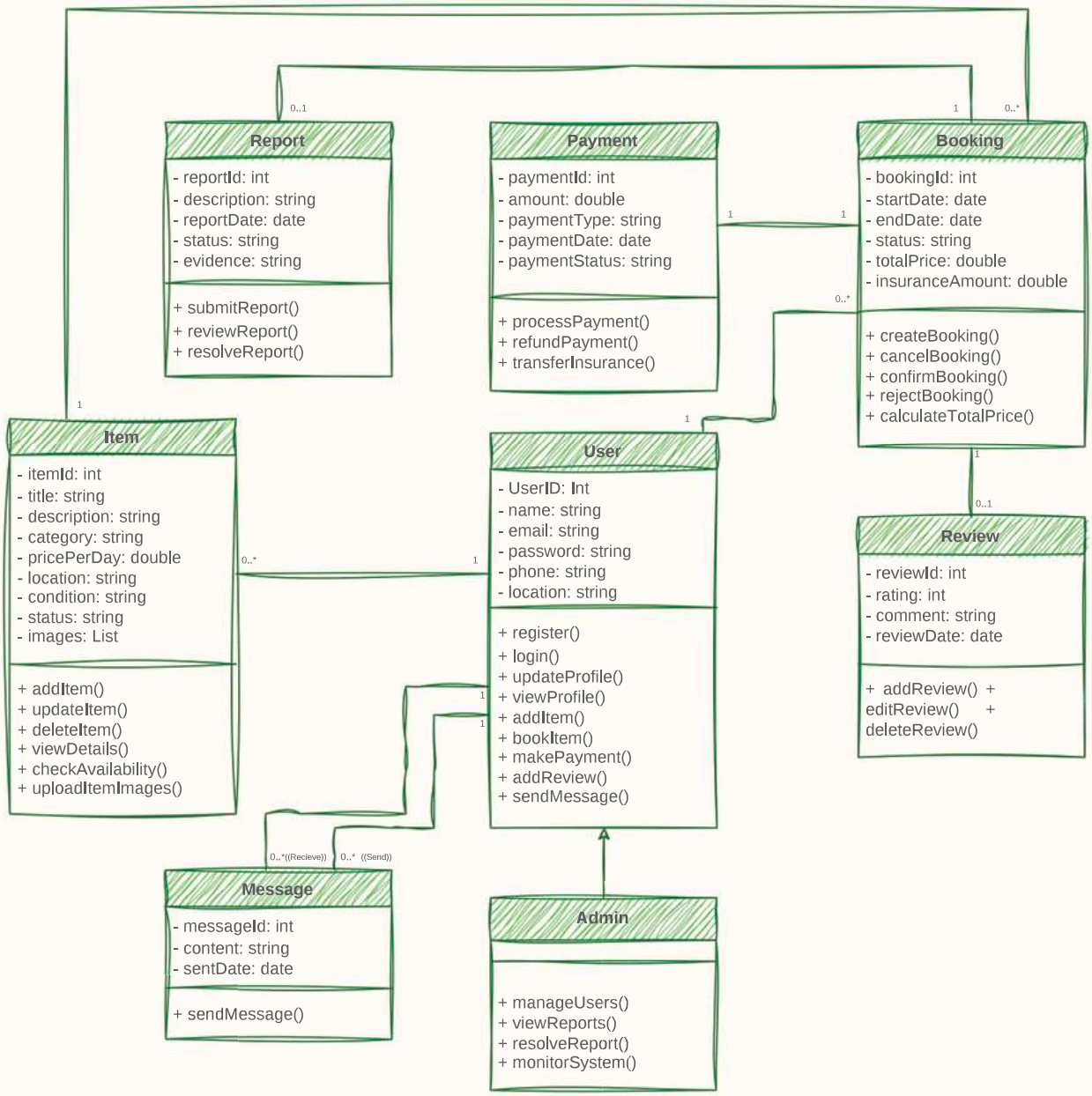
ISLAMIC UNIVERSITY OF MADINAH

RENT Right استأجر صح

3.3 Use Case Diagram

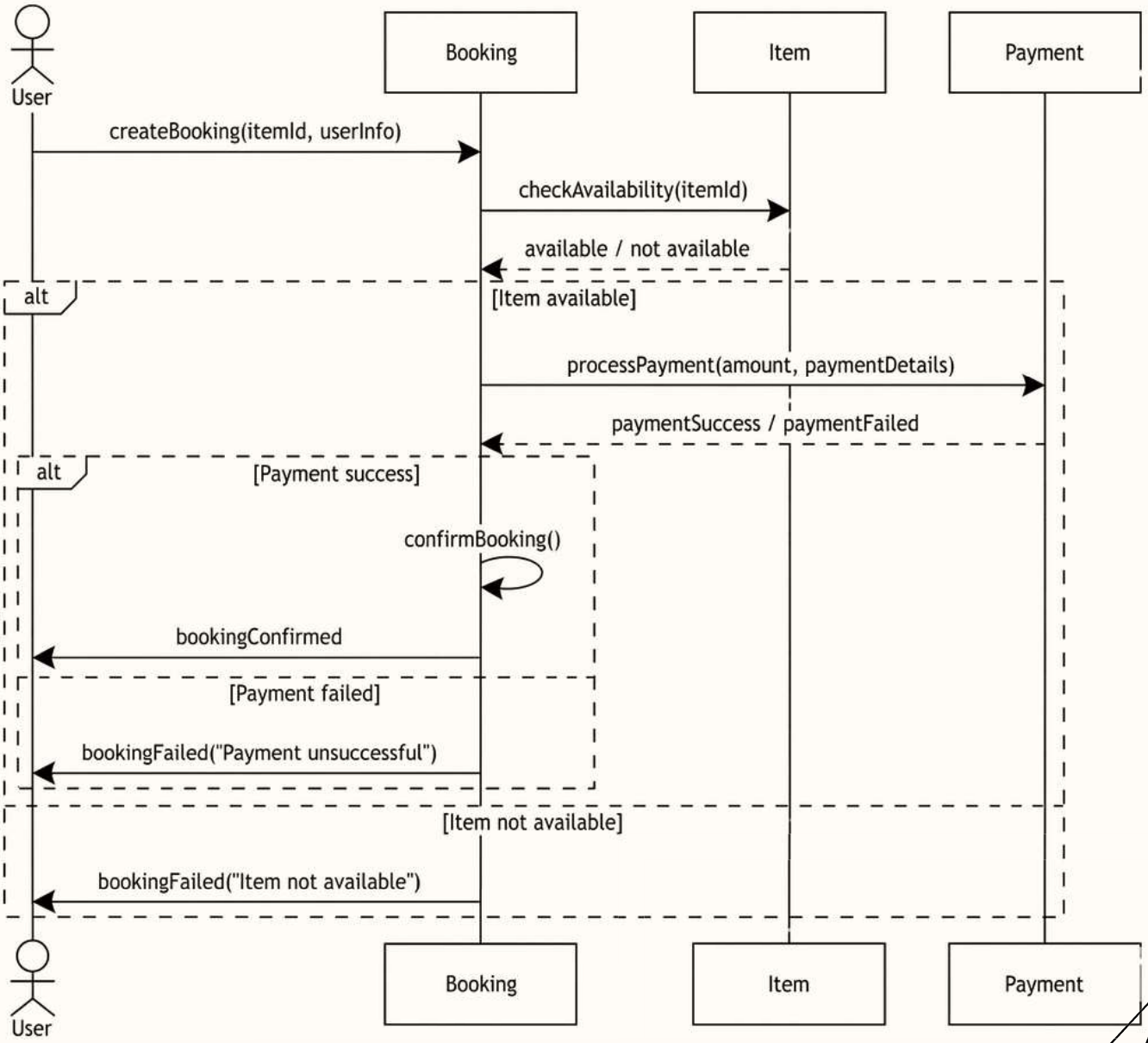


3.4 Class Diagram



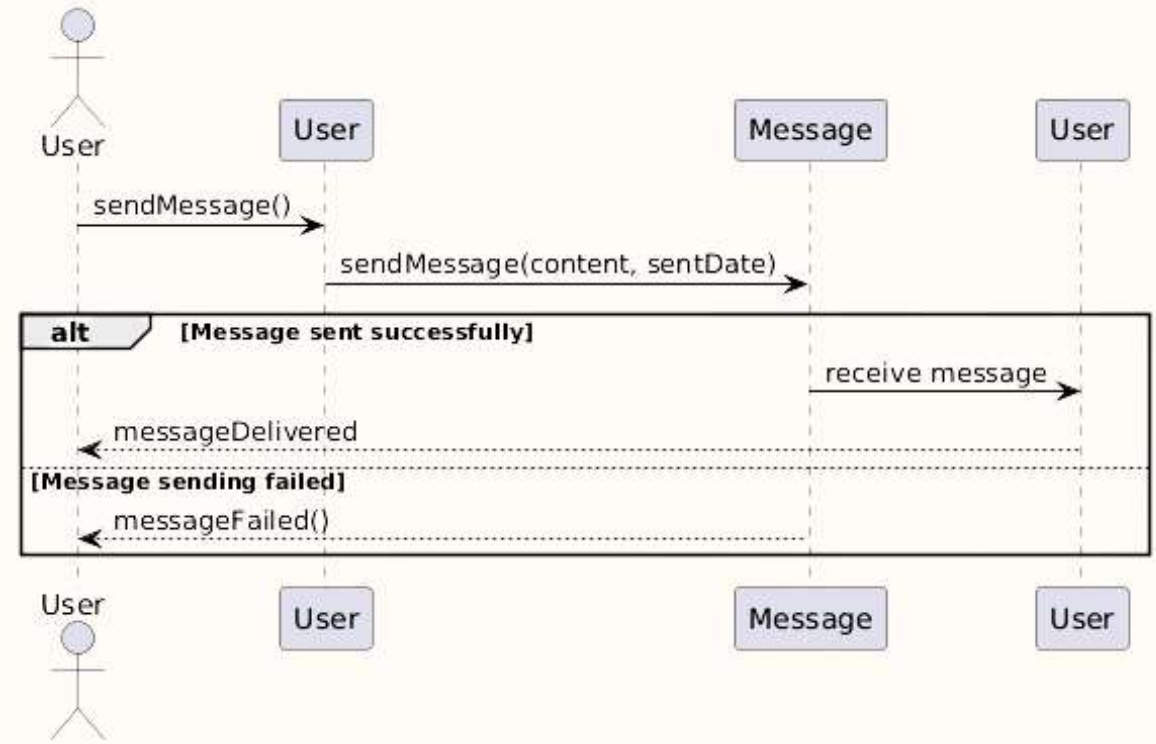
3.5 Sequence Diagram - Book an Item

This Diagram, shows the lifeline when user wants to book an item



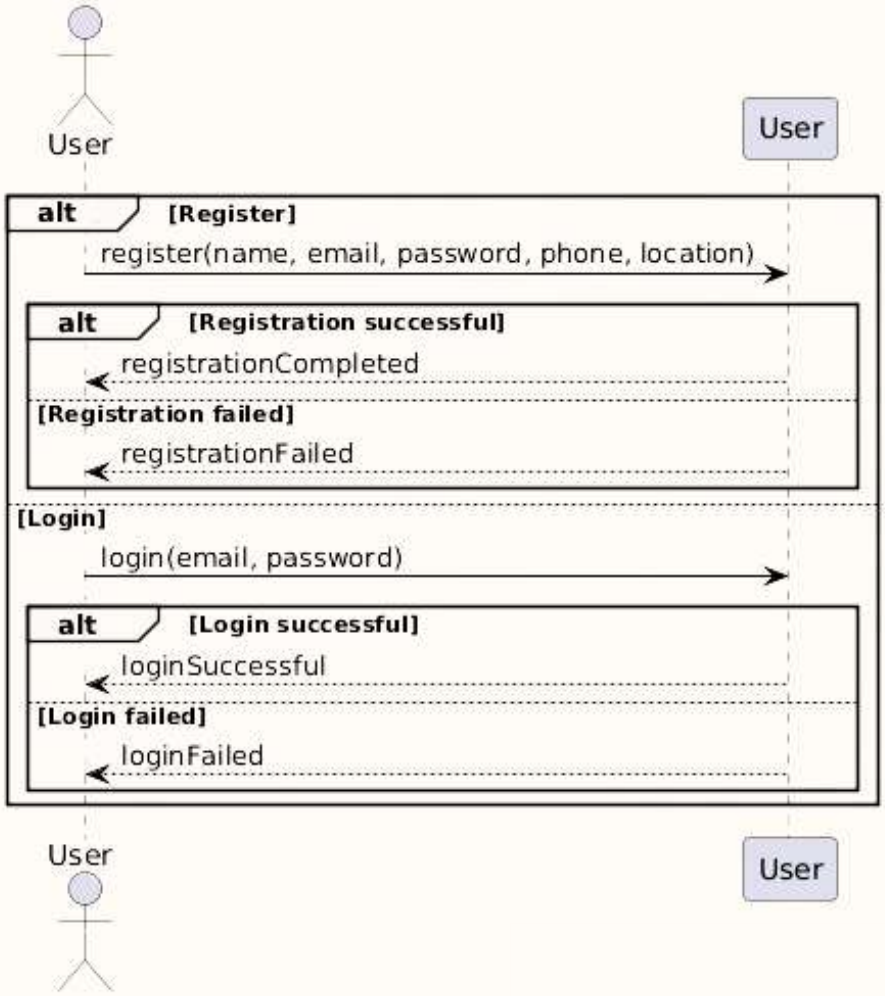
3.6 Sequence Diagram - sendMessage()

This Diagram, shows when the user want to send a message



3.7 Sequence Diagram – register() & login ()

This Diagram, shows when the user want to Register or login



3.8 Software Quality Assurance (ISO/IEC 25010 Application)

Quality assessment ensures the Rent Right system meets all user needs, both obvious and less visible, so it remains reliable and efficient. The ISO/IEC 25010 software quality model provides a way to check system quality in different areas. The following analysis shows how the system's design and construction support these qualities.

1. Maintainability

Maintainability is about how easily and quickly the system can be updated to fix issues, improve performance, or meet new requirements.
The Rent Right system is easy to maintain because it uses a Client-Server Architecture, which separates the client interface from the server logic. This strategy permits developers to update features like booking, payment, or messaging without affecting the entire system. It also makes it simpler to find and fix problems, since issues can be traced to specific parts, such as booking errors in the booking service. This design keeps the system running smoothly and makes future improvements easier.

2. Reliability

Reliability means the system works as expected under specific conditions for a certain period.
The system stays reliable by managing important tasks like booking and payments carefully. For example, it checks if an item is available before confirming a booking, which stops double bookings and keeps data correct. Payments are processed to prevent duplicates or lost information. These steps help the system run well, even with many users, and build confidence with users.

3. Usability

Usability refers to how easily and effectively users are able to achieve their goals using the system. the system.

The Rent Right system is easy to use thanks to its simple and clear interface. Users can browse, search, and book items without difficulty. The system provides clear steps and organized actions, meeting the non-functional requirement (NFR-03) for a easy-to-use interface. This helps users interact with the system comfortably and efficiently.

4. Security

The system protects data by using authentication and access control based on user roles. All users begin as general users, but some receive extra permissions to perform administrative tasks when needed. Users can act as both renters and lessors in the system, depending on whether they book or list items. Sensitive information, especially payment data, is protected through secure processing and encryption. These measures keep user data safe and meet the non-functional requirement (NFR-04) for secure authentication and transaction safety.

5. Performance Efficiency

Performance efficiency refers to how quickly the system responds and how well it uses resources in various situations.

The Rent Right system is designed for high performance, especially when users browse or search. According to (NFR-01), it provides results in less than two seconds, giving users quick responses. Effective communication between the client and server, along with proficient data management, helps the system run smoothly even with many users at the same time.

6. Portability

The system is highly portable because it is web-based, so it works on desktops, tablets, and mobile devices without any changes. This flexibility makes it easy for users to access the system from different devices and locations, without compatibility issues.